tutorial ①

Cylender has one track per surface

tracks has 4096 Sectors

given

sector = 512 byte

German University in Cairo
Faculty of Media Engineering and Technology
Spring 2025
Dr. Mohamed Karam

GUC German University in Cairo

CSEN604 – Data Bases II

Practice Assignment 1 - Solution

Exercise 1-1

Consider a hard disk with the following characteristics:

- Number of platters to read/write from: 8.
- Number of surfaces to read/write from: 16.
- Number of sectors in surface: 4096.
- Number of tracks in a surface: 2048.
- Number of bytes in a sector: 512.

If we are going to store a page in a cylinder, what is the size of a cylinder in bytes?

Solution:

A cylinder is all sectors on the same perpendicular line.
8 (platters) * 2 (surfaces) * 512 (bytes in sector) = 16*512 byte.

Exercise 1-2

You have a relation that is 5 megabytes (MB) in size. The hard disk has cylinders that are 512 kilobytes (KB) each, with each track being 128 KB in size (and all tracks have equal size).

How many cylinders are required to store the entire relation, if the storage is managed on a **cylinder-based** system? (Assume that the relation is divided into pages, with each page stored on a separate cylinder.)

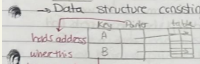
Solution:

Since 1MB = 1024 KB, then 5MB = 5120 KB.
Number of cylinders = 5120 (size of the relation) /512 (size of one cylinder) = 10 cylinders.
Note: we always take ceiling in any number division, as we always need a full page/cylinder to store the data file.

Tutorial(1)

(1) Index

→ Data structure consisting of 2 columns



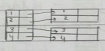
(2) Types

① Primary Index:

⇒ key is pk so it's sorted (can only have 1 primary index)

a) dense

→ record is created for every search key in the db



⇒ each record must be referenced

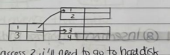
pros: fast (you can find the row fast) cons: more space (more memory)

→ I only take one copy of a repeated value

→ If i go to a repeated value, i go to the first one

b) Sparse

→ only reference a record per block



pros: less space cons: slower

can keep more index memory ↳ bec. to access 2, i'll need to go to hard disk

② Secondary Index

⇒ key is a candidate key (key that does not uniquely identify table) so it can be sorted or not ↳ no primary find

a) dense Index: Sorts data already

b) Sparse Index:

→ can only have it if it's built on a dense index as it's sorted

→ Sparse index takes some, leaves some so if i want to find Ahmed and it's not sorted, i won't be able to find all Ahmeds

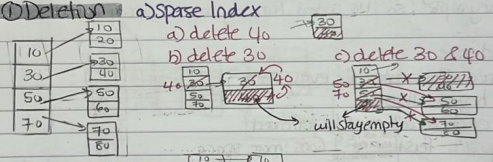
③ Multi-level

→ constructed by building an index on an index

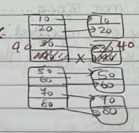
(3) Operations

① Deletion

a) Sparse Index

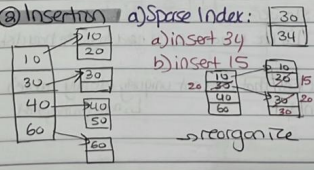


b) Dense Index



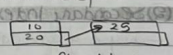
② Insertion

a) Sparse Index:



b) Dense Index → more expensive → similar to a)

⇒ we have free space



or

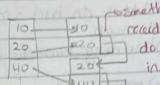
→ overflow blocks

→ reorganize later

(4) Duplicates:

a) primary index

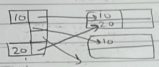
b) Secondary index



→ something/ records he'll do with indexes

① use a dense index to sort and have duplicate values in the index

⇒ excess overhead . disk space . search time

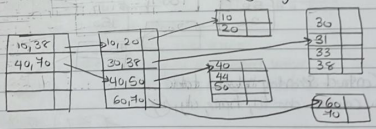
② 

→ clustering

→ problem is variable size records in index

(5) Block_Range Index (BRIN):

→ stores min_max value for every page as an index



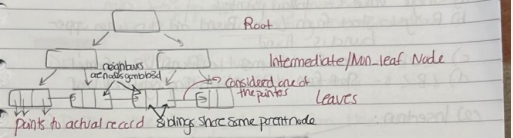
⇒ efficient in analytical queries and queries that involve narrow_range search on column w/ huge range of values

Database notes

lecture 3 B+ tree
tutorial 3

Tutorial (2)

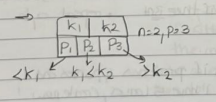
(1) B_+ Tree



→ It's an index structure and it's sorted
→ It's balanced and any path from root to any leaf has the same length

rules : • n ⇒ max no. of keys in block
• if i have n keys ⇒ n+1 pointers

→



Number of keys

	keys	Pointers	Max keys	Max Pointers	Min keys	Min Pointers
Root	n	n+1	n	n+1	1	2
Non leaf/Intermediate	n	n+1	n	n+1	$\lceil \frac{n+1}{2} \rceil - 1$	$\lceil \frac{n+1}{2} \rceil$
Leaves	n	n+1	n	n+1	$\lfloor \frac{n+1}{2} \rfloor$	$\lfloor \frac{n+1}{2} \rfloor$

to inc. branching @ non_leaf levels

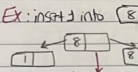
Algorithms

(1) Search
a) single value :- follow pointers
b) Range :- Start from Lower Bound and stop when upper bound is found
c) < UB :- Start from left_most leaf until UB is formed
d) LB < :- Start from LB until hitting right_most Leaf

(2) Insertion :

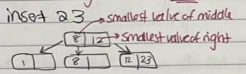
Is there num in the node?
Yes → insert Successfully!
No! → a) Split node into 2 nodes then insert
b) Adjust upper/parent node after insertion

• Ex : insert 1 into [8 | 12]



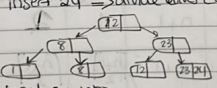
(1, (8, 12)) parent is smallest value of right subtree @ not leaf

• insert 23

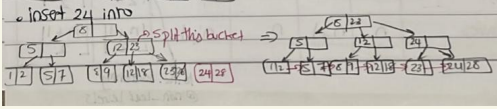


• if parent has 4 pointers and i have 5 leaves, create new parent and give it 2. our old parent get 3

• insert 24 ⇒ divide leaves equally



• insert 24 into



(3) Deletion

→ when deleting we always need to calculate min keys/pointers for leaf/non-leaf before starting ((round down))

Steps:

1) Find key pointer —then→ Follow the leaf —then→ Delete it from the file!

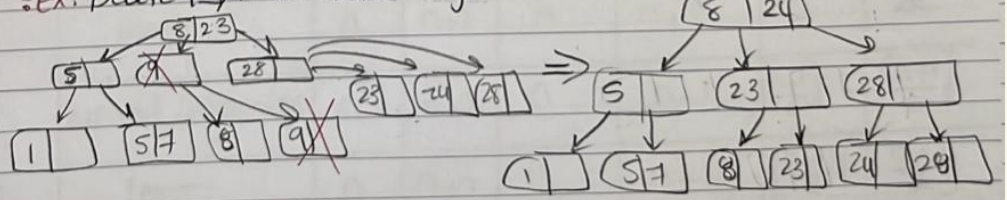
2) Delete the key_pointer pair from the tree

- if node has min no. of keys and ptrs
  ⇓ → we are done!
- if min. no. of keys and ptrs is violated
  —then→ adjust

  how? (node has some parent)

  - if a sibling is above the minimum ((left)) [Find Sibling]
    ⇒ take an extra key from the node's sibling and adjust parent
  - if sibling is at minimum
    ⇒ Merge them and adjust and delete key_ptr pair at the parent ((merge to left))

Ex: Delete 9 → redistribute = merge



tutorial (4) extensible hashing

Tutorial(3) Hashing

⇒ enhances performance of sql queries on exact value query

- Hashtables Contains Buckets / Blocks Consists of Records
  (performance: O(1) → search)

key → [Hash function h(key)] → Bucket Address / no. (where i store my record)

(1) Static Hashing

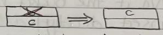
→ fixed no. of buckets ⇒ cannot split a bucket into 2

(1) Insertion

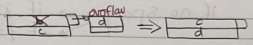
a) H(key) = Bucket No.
if bucket is full? → Add an overflow Block!

(2) Deletion:

a) Delete it from the bucket



b) After Deletion, move the lowest element in the block up!



(2) Dynamic Hashing

a) Extensible Hashing

Note: it doubles the buckets

key → [Hash function H(key)] → sequence of bits
ex: 0011 0101

higher order left most bits (per)

- Say (i) = number of bits / sequence to determine the block number used to store the key
  ⇒ we have 2^i buckets (i=1 → 2 buckets, i=2 → 4) → doubles buckets
- Say (j) = number of bits / sequence to determine the membership of the key in the bucket

Adv: can handle growing files – with less wasted space – no full reorganization

Disv: ① indirection: not best if directly in memory ② Directory doubles in size (more RAM required in disk)

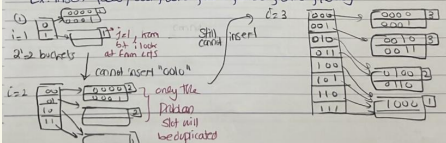
Insertion

a) h(k) = sequence of bits and use i to determine the bucket

b) No if the bucket has space then we are done, insert directly!

if no space → if j < i ⇒ split block & adjust
→ if j = i ⇒ increment i by 1
↳ duplicate buckets

Ex: insert 1000, 0001, 0010, 0011, 0100, 0110, 1000



b) Linear Hashing

Note: it inc. buckets by 1

key → [Hash fnc] → seq. of bits

Insertion case (per)

a) i = no. of bits / sequence to determine the block no. used to store the key → only most bits

b) if according to i the bucket is found → then check space → space → Done! / no space → Create Overflow bucket

⇒ if the bucket is not found
↳ the new bucket to store the key = $\boxed{old - 2^{i-1}}$
(no. of bits representing buckets; value of old bucket / where it should've been)

c) $\boxed{\text{Utilization} = \frac{\text{no. of keys used}}{\text{total no. of keys that fit}} = U}$ ⇒ Imp after every insertion

if U > threshold → inc. buckets by 1
if U ≤ threshold → we are done!
↳ given in question
↳ default: 80%

Adv: can handle growing files – less wasted space – no full reorganization
② No indirection like extensible hashing

Disadvantage: can still have overflow chains

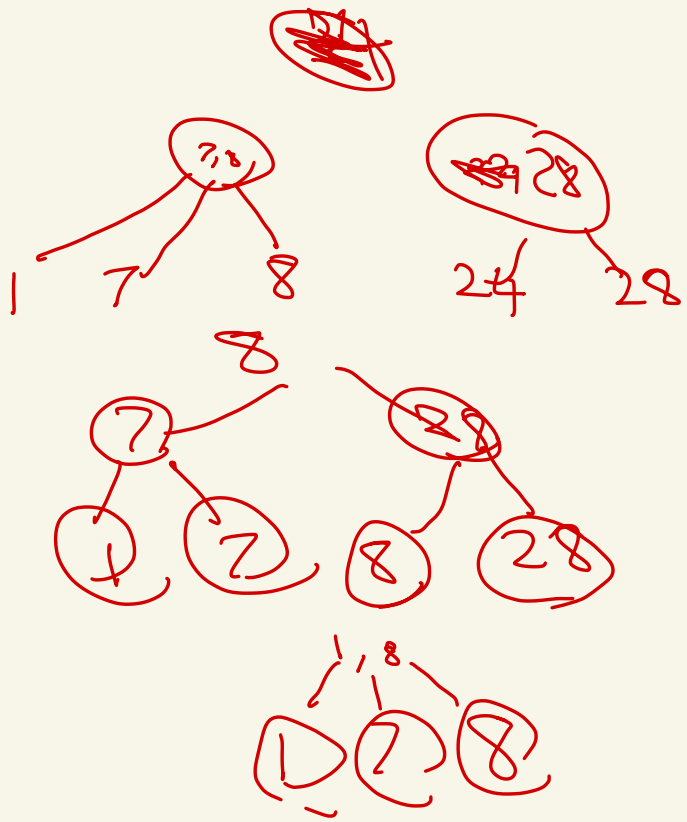
Deletion

1- simple case: JUST delete

2- coalesce with sibling: keys left in a leaf
= take from neighbor and delete node

3- Redistribute keys: take from neighbor

4- non leaf coalesc merge



tutorial 5

part hashing

HD	h(HD)
160	00
200	01
250	10
300	11

Speed	h(Speed)
1.42	000
1.86	001
2.00	010
2.10	011
2.20	100
2.66	101
2.80	110
3.20	111

00000

00 001

00 110

01 100

10 101

10 011

10 110

10 111

10 100

10 010

1003

1011

1012

1007

1001

1002

1004

1005

1008

1009

160

200

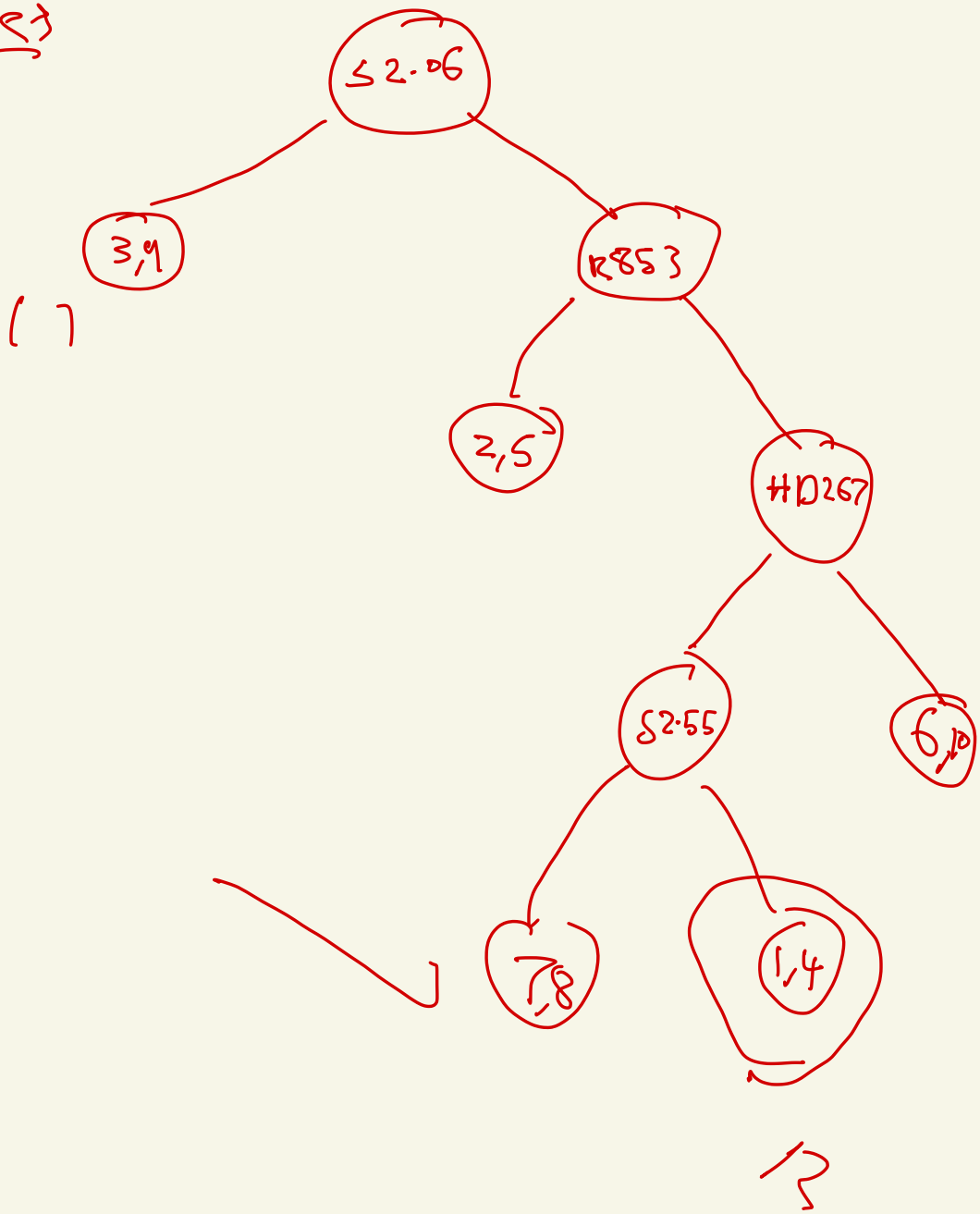
250

11111 110

Rank(70)	160	200	250	300
513	1003		1002 1005	
1024			1001, 1004	
2048				

Sri

Tree3



bitmap index

Speed	
100	0010000000011
200	0000001000000
250	1101100110000
300	0000000000100